WELCOME & MODULE INDUCTION

What we learned previously: In Module 2, we secured our "ingredients." We learned how to lock down dependencies using strict versions and cryptographic hashes, ensuring our environment builds perfectly on any server or lab machine.

What we are learning today: Now, we focus on the code itself. We are transitioning from writing "code that merely works" to writing "code that is industrially acceptable." We will cover the official Python style guide (PEP 8), Automated Linters, and Auto-Formatters.

Why this will be helpful: When you apply for a Master's program, contribute to open-source vision frameworks, or publish research alongside academic papers, reviewers will judge your code's quality before they even look at your logic. Messy formatting can cause your work to be dismissed immediately. Today, you will learn how to automate these standards so your code always looks like it was written by a senior engineer.

PEP 8 & THE COST OF "UGLY" CODE

- **The Standard:** PEP 8 is the universal style guide for Python. It dictates how to name variables, how many blank lines to use, and where to put spaces.
- **The Problem:** Memorizing every rule is inefficient and prone to human error.
- **The Goal:** Write code that is universally readable by any other Python developer instantly.

```
setOfNumbers = []

print("How many random numbers do you want to generate?")
max = int(input())

for i in range (max):
    setOfNumbers.append(random.randrange(1,101,1))
setOfNumbers.sort()
print(setOfNumbers)

print("Which number do you want to find in the set of random numbers")
searchNumber = int(input())

firstPos = 0
lastPos = max-1
found = False

while (not found and firstPos <= lastPos):
    midPos = int((firstPos + lastPos)/2)

    if (searchNumber == setOfNumbers[midPos]) :
        found = True
    else :
        if (searchNumber < setOfNumbers[midPos]):
            lastPos = midPos - 1
        else :
            firstPos = midPos + 1


if (found) :
    print("Your item is in the list")
else :
    print("Your item is not in the list")
```

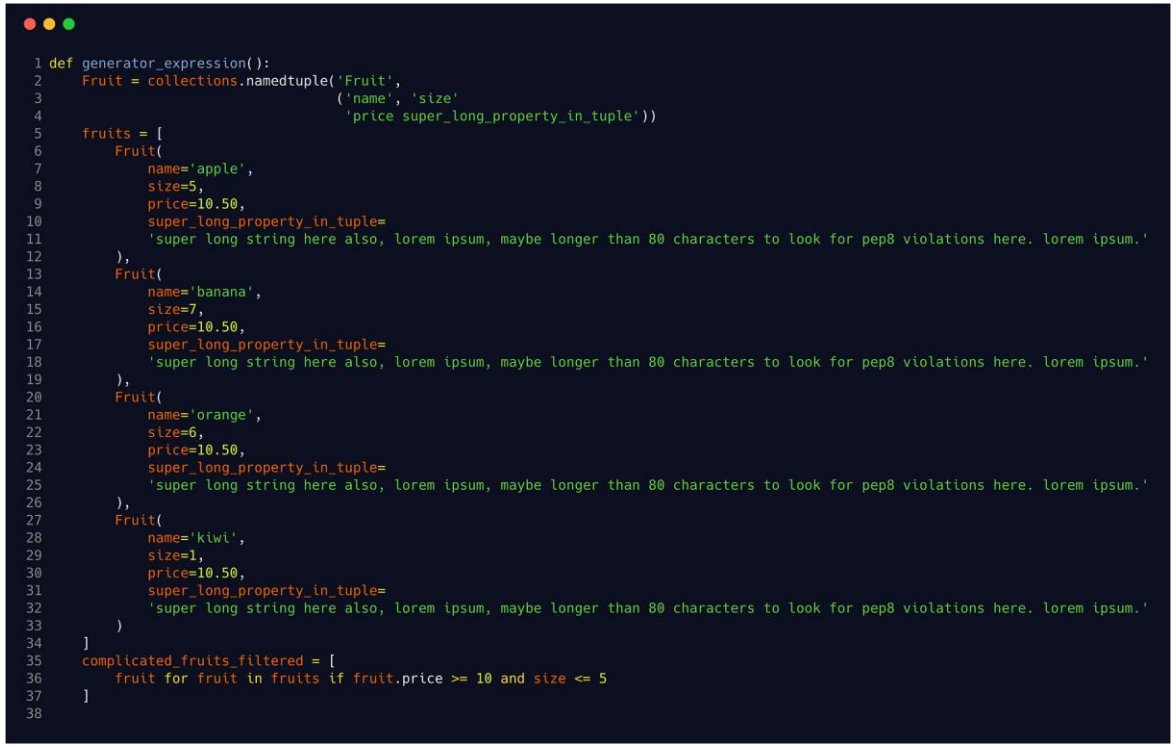
AUTOMATED LINTERS (FLAKE8 & PYLINT)

What is a Linter? Think of it as a strict spell-checker for logic and style.

How it works: It performs *static analysis*. It reads your code without running it and flags stylistic errors, undefined variables, and overly complex functions before you even hit "execute."

Problem	Pylint output	Flake8 output	PyFlakes output
unnecessary semicolon	✓	✓	✗
missing newlines	✓	✓	✗
missing whitespaces	✗	✓	✗
missing docstring	✓	✗	✗
unused arguments	✓	✓	✓
incorrect naming style	✓	✗	✗
selecting element from the list with incorrect index	✗	✗	✗
dividing by zero expression	✗	✗	✗
missing return statement	✓	✗	✗
calling method that doesn't exist	✓	✓	✓
unnecessary pass statement	✓	✗	✗

THE "SENIOR ENGINEER" BUTTON: AUTO-FORMATTERS



```python
def generator_expression():
    Fruit = collections.namedtuple('Fruit',
                                   ('name', 'size'
                                    'price super_long_property_in_tuple'))
    fruits = [
        Fruit(
            name='apple',
            size=5,
            price=10.50,
            super_long_property_in_tuple=
            'super long string here also, lorem ipsum, maybe longer than 80 characters to look for pep8 violations here. lorem ipsum.'
        ),
        Fruit(
            name='banana',
            size=7,
            price=10.50,
            super_long_property_in_tuple=
            'super long string here also, lorem ipsum, maybe longer than 80 characters to look for pep8 violations here. lorem ipsum.'
        ),
        Fruit(
            name='orange',
            size=6,
            price=10.50,
            super_long_property_in_tuple=
            'super long string here also, lorem ipsum, maybe longer than 80 characters to look for pep8 violations here. lorem ipsum.'
        ),
        Fruit(
            name='kiwi',
            size=1,
            price=10.50,
            super_long_property_in_tuple=
            'super long string here also, lorem ipsum, maybe longer than 80 characters to look for pep8 violations here. lorem ipsum.'
        )
    ]
    complicated_fruits_filtered = [
        fruit for fruit in fruits if fruit.price >= 10 and size <= 5
    ]

```

Finding vs. Fixing: Finding errors with a linter is great, but fixing them manually wastes time.

isort: Automatically groups and alphabetizes your imports at the top of the file so they look organized.

Black: Known as the "uncompromising code formatter." It takes your messy code, breaks it down, and completely rewrites it to perfect PEP 8 standards in milliseconds.